

Attribute#

<https://pytorch.org/docs/stable/generated/torch.jit.Attribute.html#torch.jit>

Description:

This method is a pass-through function that returns value, mostly used to indicate to the TorchScript compiler that the left-hand side expression is a class instance attribute with type of type. Note that `torch.jit.Attribute` should only be used in `__init__` method of `jit.ScriptModule` subclasses. Though TorchScript can infer correct type for most Python expressions, there are some cases where type inference can be wrong, including: Empty containers like `[]` and `{}`, which TorchScript assumes to be container of Tensor Optional types like `Optional[T]` but assigned a valid value of type `T`, TorchScript would assume it is type `T` rather than `Optional[T]` In eager mode, it is simply a pass-through function that returns value without other implications.

Function Signature

```
class torch.jit.Attribute(value, type)#
```

Parameters

Returns

```
count(value, /)#
```

```
index(value, start=0, stop=9223372036854775807, /)#
```

Returns:

Returns value

Code Examples

```

import torch
from typing import Dict

class AttributeModule(torch.jit.ScriptModule):
    def __init__(self) -> None:
        super().__init__()
        self.foo = torch.jit.Attribute(0.1, float)

        # we should be able to use self.foo as a float here
        assert 0.0 < self.foo

        self.names_ages = torch.jit.Attribute({}, Dict[str,
int])

        self.names_ages["someone"] = 20
        assert isinstance(self.names_ages["someone"], int)

m = AttributeModule()
# m will contain two attributes
# 1. foo of type float
# 2. names_ages of type Dict[str, int]

```

```

import torch
from typing import Dict

class AttributeModule(torch.nn.Module):
    names: Dict[str, int]

    def __init__(self) -> None:
        super().__init__()
        self.names = {}

m = AttributeModule()

```

Detailed Documentation

Documentation

This method is a pass-through function that returns value, mostly used to indicate to the TorchScript compiler that the left-hand side expression is a class instance attribute with type of type. Note that `torch.jit.Attribute` should only be used in `__init__` method of `jit.ScriptModule` subclasses.

Though TorchScript can infer correct type for most Python expressions, there are some cases where type inference can be wrong, including:

Empty containers like `[]` and `{}`, which TorchScript assumes to be container of Tensor

Optional types like `Optional[T]` but assigned a valid value of type `T`, TorchScript would assume it is type `T` rather than `Optional[T]`

In eager mode, it is simply a pass-through function that returns value without other implications.

Note: it's now preferred to instead use type annotations instead of `torch.jit.Attribute`:

value – An initial value to be assigned to attribute.

Return number of occurrences of value.

Return first index of value.

Raises `ValueError` if the value is not present.

Alias for field number 1

Alias for field number 0